

บทที่ 2

คำสั่งของเจเอสพี

(JSP Commands)

เนื้อหา

- องค์ประกอบหลักของไฟล์ JSP
- คำสั่งใน JSP (JavaServer Pages)
- Directives (คำสั่งกำกับไฟล์ JSP)
- Declarations (ตัวแปรและเมธอดของ Java)
- Scriptlets (โค้ด Java ผังใน JSP)
- Expressions (แสดงค่าของตัวแปรหรือเมธอด)
- Implicit Objects (อ็อบเจกต์ที่สามารถเรียกใช้ได้โดยตรง)
- Standard Actions (คำสั่งมาตรฐานของ JSP)
- JSTL (JavaServer Pages Standard Tag Library)
- Expression Language (EL)
- Custom Tags (แท็กที่สร้างขึ้นเอง)
- ตัวอย่างเพิ่มเติม ของคำสั่งต่างๆ ใน JSP
- คำสั่ง JSP ที่สามารถใช้งานใน NetBeans IDE รันบน Glassfish Web Server
- JSTL (JavaServer Pages Standard Tag Library)
- JavaBeans, JSTL, EL (Expression Language) และ การจัดการกับข้อผิดพลาด
- คำสั่งรับข้อมูลใน JSP (JSP Input Handling) และรายละเอียดแต่ละประเภท
- เพิ่มวิธีการรับข้อมูลใน JSP อย่างละเอียดเพิ่มเติม
- โปรเจกต์ JSP ที่มี Homepage ใช้ NetBeans + GlassFish Web Server

ในยุคที่เทคโนโลยีเว็บมีการพัฒนาอย่างรวดเร็ว การสร้างเว็บไซต์แบบไดนามิกและตอบสนองความต้องการของผู้ใช้มีประสิทธิภาพเป็นสิ่งจำเป็น JSP (JavaServer Pages) จึงกลายเป็นเครื่องมือสำคัญในการพัฒนาเว็บแอปพลิเคชันบนพื้นฐานของภาษา Java โดยเฉพาะเมื่อใช้งานร่วมกับเครื่องมือพัฒนาอย่าง NetBeans IDE และ GlassFish Web Server ผู้พัฒนาเว็บสามารถสร้างระบบที่มีโครงสร้างดี มีความปลอดภัย และสามารถขยายตัวได้ในระยะยาว หนังสือเล่มนี้จึงมุ่งเน้นการอธิบายองค์ประกอบหลักของ JSP อย่างเป็นระบบ พร้อมตัวอย่างที่สามารถประยุกต์ใช้ได้จริงในโปรเจกต์

เนื้อหาในเล่มจะเริ่มต้นด้วยการแนะนำ องค์ประกอบหลักของไฟล์ JSP ซึ่งรวมถึง Directives หรือคำสั่งกำกับไฟล์ที่ใช้กำหนดลักษณะของหน้า JSP, Declarations ที่ใช้สร้างตัวแปรและเมธอด Java ภายในหน้า JSP, Scriptlets ที่ฝังโค้ด Java ไว้ในส่วนของ HTML, และ Expressions ที่ใช้แสดงค่าผลลัพธ์จากตัวแปรหรือเมธอดโดยตรง โดยเนื้อหาทั้งหมดนี้จะถูกอธิบายอย่างละเอียดและเสริมด้วยตัวอย่างที่สามารถทดสอบได้ใน NetBeans

นอกจากนี้ ยังได้กล่าวถึงการใช้งาน **Implicit Objects** ที่ JSP เตรียมไว้ให้ใช้งานโดยอัตโนมัติ เช่น request, response, session, application เป็นต้น รวมถึง **Standard Actions** อย่าง `<jsp:include>` และ `<jsp:forward>` ที่ช่วยในการควบคุมโฟลว์ของเว็บเพจได้อย่างยืดหยุ่น เนื้อหาจะขยายความไปถึงการใช้ **JSTL (JavaServer Pages Standard Tag Library)** และ **Expression Language (EL)** เพื่อเขียนโค้ดที่อ่านง่าย แยกส่วนการประมวลผลออกจากส่วนแสดงผล และลดการใช้ Scriptlets ที่มักทำให้โค้ดยุ่งยากในการบำรุงรักษา

อีกส่วนสำคัญที่ไม่อาจมองข้ามคือ **Custom Tags** หรือการสร้างแท็ก JSP ขึ้นเอง ซึ่งช่วยให้สามารถกำหนดพฤติกรรมซ้ำ ๆ ในเว็บเพจอย่างเป็นระบบ รวมถึงการใช้ **JavaBeans** ร่วมกับ JSTL และ EL เพื่อแยกตรรกะธุรกิจจากการแสดงผล และยังมีเนื้อหาเกี่ยวกับการจัดการข้อผิดพลาด (Exception Handling) ใน JSP เพื่อให้ระบบมีความเสถียรและปลอดภัยยิ่งขึ้น ทั้งยังกล่าวถึงเทคนิคการรับข้อมูลจากผู้ใช้ด้วยวิธีต่าง ๆ ทั้งแบบ GET, POST, ผ่าน Form Elements หรือแม้แต่ Ajax บน JSP

นอกจากนั้นยังได้รวมตัวอย่างโปรเจกต์ JSP ที่มี **Homepage** แบบครบวงจร พัฒนาด้วย **NetBeans IDE** และ **GlassFish Web Server** พร้อมทั้งคำอธิบายแบบ Step-by-Step ตั้งแต่การสร้างหน้าแรก การรับข้อมูล การประมวลผล ไปจนถึงการแสดงผลแบบมีเงื่อนไขและมีโครงสร้าง MVC ครบถ้วน โดยหวังว่าผู้อ่านจะสามารถนำความรู้จากหนังสือเล่มนี้ไปประยุกต์ใช้ในการพัฒนาเว็บแอปพลิเคชัน JSP ได้อย่างมีประสิทธิภาพ

องค์ประกอบหลักของไฟล์ JSP

ไฟล์ .jsp ประกอบไปด้วยส่วนสำคัญดังต่อไปนี้:

1. **Directives** (คำสั่งกำกับไฟล์ JSP)
2. **Declarations** (ตัวแปรและเมธอดของ Java)
3. **Scriptlets** (โค้ด Java ฝังใน JSP)
4. **Expressions** (แสดงค่าของตัวแปรหรือเมธอด)
5. **Implicit Objects** (อ็อบเจกต์ที่สามารถเรียกใช้ได้โดยตรง)
6. **Standard Actions** (คำสั่งมาตรฐานของ JSP)
7. **JSTL (JavaServer Pages Standard Tag Library)**
8. **Expression Language (EL)**

9. Custom Tags (แท็กที่สร้างขึ้นเอง)

1. Directives (คำสั่งกำกับไฟล์ JSP)

Directives เป็นคำสั่งที่ใช้กำหนดคุณสมบัติของไฟล์ JSP เช่น การกำหนดประเภทของไฟล์, การนำเข้าแพ็คเกจ Java, และการตั้งค่าต่าง ๆ

รูปแบบของ Directives

```
<%@ directive attribute="value" %>
```

ประเภทของ Directives

1. **Page Directive** – กำหนดคุณสมบัติของไฟล์ JSP
2. `<%@ page language="java" contentType="text/html; charset=UTF-8" %>`
3. **Include Directive** – ใ้รวมไฟล์ JSP หรือ HTML อื่นเข้ามา
4. `<%@ include file="header.jsp" %>`
5. **Taglib Directive** – ใช้กำหนดไลบรารีแท็กที่ต้องการใช้งาน
6. `<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>`

2. Declarations (ตัวแปรและเมธอดของ Java)

ใช้สำหรับประกาศตัวแปรและเมธอดใน JSP ซึ่งจะถูกสร้างเป็นฟิลด์หรือเมธอดในคลาสที่ถูกคอมไพล์

รูปแบบของ Declarations

```
<%! dataType variableName; %>
```

```
<%! returnType methodName() { ... } %>
```

ตัวอย่าง

```
<%! int count = 0; %>
```

```
<%! int increment() { return ++count; } %>
```

3. Scriptlets (โค้ด Java ผังใน JSP)

ใช้เขียนโค้ด **Java** ที่ต้องการให้รันภายในไฟล์ JSP

รูปแบบของ Scriptlets

```
<% Java code %>
```

ตัวอย่าง

```
<%
```

```
    String name = "John";
```

```
    int age = 25;
```

```
%>
```

```
<p>Name: <%= name %></p>
```

<p>Age: <%= age %></p>

4. Expressions (แสดงค่าของตัวแปรหรือเมธอด)

ใช้แสดงผลค่าของตัวแปรหรือเมธอดโดยตรงในหน้าเว็บ

รูปแบบของ **Expressions**

<%= expression %>

ตัวอย่าง

<p>Current Time: <%= new java.util.Date() %></p>

5. Implicit Objects (อ็อบเจกต์ที่ใช้ได้โดยตรง)

JSP มี **implicit objects** ที่สามารถเรียกใช้ได้โดยไม่ต้องประกาศ ซึ่งรวมถึง:

- request → ใช้ดึงข้อมูลที่ส่งมาจาก **client**
- response → ใช้กำหนดค่าที่จะส่งกลับไปยัง **client**
- session → ใช้จัดเก็บข้อมูลของผู้ใช้ขณะยังอยู่ใน session
- application → ใช้จัดเก็บข้อมูลของแอปพลิเคชัน
- out → ใช้แสดงผลข้อมูลไปยัง **client**

ตัวอย่าง

<p>Client IP: <%= request.getRemoteAddr() %></p>

6. Standard Actions (คำสั่งมาตรฐานของ JSP)

JSP มีชุดคำสั่งที่ช่วยให้สามารถทำงานร่วมกับ **JavaBeans**, **Forward Requests**, และ รวมไฟล์ JSP อื่นๆ

ตัวอย่างคำสั่ง **Standard Actions**

1. ใช้ **JavaBean**
2. <jsp:useBean id="user" class="com.example.User" scope="session"/>
3. <jsp:setProperty name="user" property="name" value="John Doe"/>
4. <jsp:getProperty name="user" property="name"/>
5. **Redirect** ไปยังไฟล์อื่น
6. <jsp:forward page="home.jsp"/>

7. JSTL (JavaServer Pages Standard Tag Library)

JSTL คือไลบรารีแท็กมาตรฐานของ JSP ที่ช่วยลดการใช้ **Java code** ภายใน JSP และทำให้โค้ดสะอาดขึ้น

ตัวอย่าง

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

```
<c:if test="${param.username == 'admin'}">
```

```
    <p>Welcome, Admin!</p>
```

```
</c:if>
```

8. Expression Language (EL)

Expression Language หรือ **EL** เป็นวิธีที่ง่ายกว่าในการเข้าถึงข้อมูลใน **request**, **session**, **application** โดยไม่ต้องใช้ Java code

ตัวอย่าง

```
<p>Username: ${sessionScope.user}</p>
```

```
<p>Product Price: ${product.price}</p>
```

9. Custom Tags (แท็กที่สร้างขึ้นเอง)

เราสามารถสร้าง **Custom Tag** ขึ้นมาเองได้โดยใช้ **Tag Library Descriptor (TLD)** และ **Java Class**

ตัวอย่าง

```
<%@ taglib prefix="myTag" uri="WEB-INF/myTags.tld" %>
```

```
<myTag:hello name="John"/>
```

สรุป

องค์ประกอบ	คำอธิบาย	ตัวอย่าง
Directives	คำสั่งกำกับไฟล์ JSP	<%@ page language="java" %>
Declarations	ประกาศตัวแปร/เมธอด	<%! int count = 0; %>
Scriptlets	ฝังโค้ด Java	<% int x = 10; %>
Expressions	แสดงค่าตัวแปร	<%= new java.util.Date() %>
Implicit Objects	อ็อบเจกต์ใน JSP	<%= request.getRemoteAddr() %>
Standard Actions	คำสั่งมาตรฐาน JSP	<jsp:forward page="home.jsp"/>
JSTL	ไลบรารีแท็กของ JSP	<c:if test="\${user == 'admin'}">
Expression Language (EL)	ใช้แสดงค่าตัวแปร	\${sessionScope.user}
Custom Tags	แท็กที่สร้างขึ้นเอง	<myTag:hello name="John"/>

องค์ประกอบทั้งหมดนี้เป็นพื้นฐานของ **JSP** ที่สามารถนำไปใช้พัฒนา **Web Application** ที่มีความซับซ้อนขึ้นได้

คำสั่งใน JSP (JavaServer Pages)

คำสั่งใน **JSP (JavaServer Pages)** สามารถแบ่งออกเป็น 3 ประเภทหลักๆ ได้แก่ **Directives**, **Declarations**, และ **Scriptlets** ซึ่งแต่ละประเภทมีการใช้งานและจุดประสงค์ที่แตกต่างกันไป เพื่อช่วยในการเขียนโค้ด JSP ที่มีประสิทธิภาพและสามารถทำงานได้ตามความต้องการ ต่อไปนี้จะอธิบายแต่ละประเภทพร้อมตัวอย่างโดยละเอียด:

1. Directives (คำสั่งกำหนดทิศทาง)

Directives ใช้สำหรับการกำหนดค่าคอนฟิกหรือการตั้งค่าต่างๆ ที่จะมีผลในระดับของทั้งหน้า JSP โดยทั่วไปจะใช้ในการกำหนดค่า เช่น การตั้งค่า **content type**, การกำหนด **import class**, หรือการตั้งค่า **tag libraries** เป็นต้น

ประเภทของ **Directives**:

- **Page Directive** (<%@ page ... %>)
- **Include Directive** (<%@ include ... %>)
- **Taglib Directive** (<%@ taglib ... %>)

1.1 Page Directive

คำสั่ง **Page Directive** ใช้ในการตั้งค่าหรือปรับแต่งพฤติกรรมของหน้า JSP เช่น กำหนด **content type**, **language**, หรือ **import** ไฟล์ Java

ตัวอย่าง:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" import="java.util.*" %>
<html>
  <head>
    <title>Page Directive Example</title>
  </head>
  <body>
    <h1>Welcome to the JSP page with Page Directive</h1>
  </body>
</html>
```

ในตัวอย่างนี้:

- **contentType** กำหนดรูปแบบของเนื้อหาของหน้าเป็น **HTML** พร้อมด้วย **charset=UTF-8**
- **language** กำหนดว่าใช้ภาษา **Java** ในการเขียนโค้ด
- **import** ใช้เพื่อ **import** คลาสจาก **Java API** ที่ต้องการใช้งานในหน้า JSP

1.2 Include Directive

คำสั่ง **Include Directive** ใช้สำหรับการรวมไฟล์อื่นเข้ามาในหน้า JSP ตัวนี้จะช่วยให้สามารถรวมไฟล์ที่ต้องการซ้ำในหลายๆ หน้า เช่น ฟุตเตอร์หรือเฮดเดอร์

ตัวอย่าง:

```
<%@ include file="header.jsp" %>
<html>
  <body>
    <h2>This is the body of the page</h2>
  </body>
</html>
```

```
<%@ include file="footer.jsp" %>
```

ในตัวอย่างนี้จะรวม **header.jsp** และ **footer.jsp** เข้าไปในไฟล์ JSP หลัก โดยที่โค้ดในไฟล์ที่ถูก

include จะถูกนำมาวางตรงจุดที่คำสั่ง **include** ถูกเขียน

1.3 Taglib Directive

คำสั่ง **Taglib Directive** ใช้สำหรับการประกาศ **JSP tag libraries** ซึ่งจะช่วยให้เราสามารถใช้งานแท็กที่กำหนดเองได้ใน JSP

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <body>
    <h2>Item List:</h2>
    <c:forEach var="item" items="${items}">
      <p>${item}</p>
    </c:forEach>
  </body>
</html>
```

ในตัวอย่างนี้:

- **taglib** จะประกาศการใช้งาน **JSTL core library** โดยที่ **prefix** เป็น **c**
- ใช้ **<c:forEach>** ในการวนลูปแสดงรายการของ **items**

2. Declarations (คำสั่งการประกาศ)

Declarations ใช้ในการประกาศตัวแปรหรือเมธอดใน JSP ซึ่งการประกาศในส่วนนี้จะมีผลในระดับของทั้งหน้า JSP และสามารถใช้ในการเขียนโค้ดที่ซับซ้อน เช่น การประกาศฟังก์ชันหรือคลาสในหน้า JSP

ตัวอย่าง:

```
<%!
```

```

    public String greet(String name) {
        return "Hello, " + name + "!";
    }
%>

```

```

<html>
  <body>
    <h2><%= greet("John") %></h2>
  </body>
</html>

```

ในตัวอย่างนี้:

- **<%! %>** ใช้ในการประกาศ **Java method** ที่สามารถเรียกใช้ในหน้า JSP ได้
- เมธอด **greet(String name)** จะรับชื่อผู้ใช้และแสดงข้อความทักทาย
- ในส่วนของ **<%= %>** จะใช้เรียกเมธอด **greet** และแสดงผลในหน้า JSP

3. Scriptlets (คำสั่งสคริปต์)

Scriptlets ใช้ในการเขียนโค้ด **Java** ภายใน JSP โดยโค้ดใน **scriptlet** จะถูกประมวลผลในระหว่างการทำงานของ JSP และสามารถใช้ในการคำนวณหรือประมวลผลต่างๆ ที่จำเป็นต้องทำในหน้าดังกล่าว ตัวอย่าง:

```

<html>
  <head>
    <title>Scriptlet Example</title>
  </head>
  <body>
    <h2>
      <%
        int a = 5;
        int b = 10;
        int sum = a + b;
        out.println("The sum of " + a + " and " + b + " is " + sum);
      %>
    </h2>
  </body>
</html>

```

ในตัวอย่างนี้:

- ใช้ **scriptlet** ในการคำนวณผลรวมของตัวเลขสองตัว และแสดงผลลัพธ์ออกมาบนหน้าเว็บ
- โค้ดที่อยู่ใน `<% %>` จะถูกประมวลผลก่อนที่จะส่งไปยังผู้ใช้

สรุป

1. **Directives** (คำสั่งกำหนดทิศทาง): ใช้สำหรับการกำหนดค่าต่าง ๆ ที่มีผลในระดับของทั้งหน้า JSP เช่น **page**, **include**, และ **taglib**
2. **Declarations** (คำสั่งการประกาศ): ใช้ในการประกาศตัวแปรหรือเมธอดที่สามารถใช้ในหน้า JSP
3. **Scriptlets** (คำสั่งสคริปต์): ใช้ในการเขียนโค้ด **Java** ภายใน JSP เพื่อประมวลผลหรือคำนวณค่าต่าง ๆ ที่ต้องการในหน้าเว็บ

แต่ละประเภทมีบทบาทและจุดประสงค์ที่แตกต่างกันไปในการช่วยพัฒนาแอปพลิเคชัน JSP ให้ง่ายและมีประสิทธิภาพมากขึ้น

ใน **JSP (JavaServer Pages)** มีคำสั่ง (commands) ที่สามารถใช้ได้หลากหลาย ซึ่งแต่ละคำสั่งมีประโยชน์ในการจัดการกับข้อมูล, แสดงผล, และการทำงานต่าง ๆ ภายในหน้า JSP ต่อไปนี้คือ **command list** ของ JSP พร้อมประโยชน์และตัวอย่างการใช้งาน:

1. Directive Commands

คำสั่งในประเภทนี้ใช้สำหรับการกำหนดค่าคอนฟิกหรือการตั้งค่าพฤติกรรมของหน้า JSP โดยจะมีผลในระดับของทั้งไฟล์ JSP

1.1 Page Directive

ประโยชน์: ใช้ในการตั้งค่าคอนฟิกทั่วไปของหน้า JSP เช่น **contentType**, **language**, **import**, หรือ **buffering**

ตัวอย่าง:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" import="java.util.*" %>
<html>
  <head>
    <title>Page Directive Example</title>
  </head>
  <body>
    <h1>Page Directive Example</h1>
  </body>
</html>
```

- **contentType**: กำหนดประเภทของข้อมูล (MIME type) ของหน้า เช่น **text/html** และ **charset=UTF-8**

- **language:** กำหนดภาษาที่ใช้ในหน้า JSP (ในที่นี้คือ **Java**)
- **import:** ใช้ในการนำเข้า **Java classes** ที่ต้องการใช้งานใน JSP

1.2 Include Directive

ประโยชน์: ใช้สำหรับการรวมไฟล์ JSP อื่นเข้ามาในหน้า JSP นี้ ซึ่งมักใช้สำหรับการรวมฟุตเตอร์หรือเฮดเดอร์ที่ซ้ำกันในหลายๆ หน้า

ตัวอย่าง:

```
<%@ include file="header.jsp" %>
```

```
<html>
```

```
  <body>
```

```
    <h1>Main Content</h1>
```

```
  </body>
```

```
</html>
```

```
<%@ include file="footer.jsp" %>
```

ในตัวอย่างนี้จะรวมไฟล์ **header.jsp** และ **footer.jsp** เข้ามาในหน้า JSP หลัก

1.3 Taglib Directive

ประโยชน์: ใช้เพื่อประกาศการใช้งาน **JSP tag libraries** (เช่น **JSTL**) ที่ช่วยให้สามารถใช้แท็กที่กำหนดเองได้ใน JSP

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

```
<html>
```

```
  <body>
```

```
    <c:set var="items" value="${['Apple', 'Banana', 'Cherry']}" />
```

```
    <c:forEach var="item" items="${items}">
```

```
      <p>${item}</p>
```

```
    </c:forEach>
```

```
  </body>
```

```
</html>
```

- **taglib** ประกาศการใช้งาน **JSTL** core library โดยที่ **prefix** เป็น **c**
- ใช้แท็ก **<c:forEach>** เพื่อวนลูปรายการผลไม้

2. Scripting Elements

ประโยชน์: ใช้ในการเขียนโค้ด **Java** ภายในหน้า JSP เพื่อทำการประมวลผลข้อมูลที่ต้องการหรือการแสดงผล

2.1 Scriptlet

ประโยชน์: ใช้ในการเขียนโค้ด **Java** ที่จะประมวลผลและแสดงผลในหน้า JSP

ตัวอย่าง:

```
<html>
  <body>
    <h2>
      <%
        int a = 5;
        int b = 10;
        int sum = a + b;
        out.println("The sum of " + a + " and " + b + " is " + sum);
      %>
    </h2>
  </body>
</html>
```

- **Scriptlet** ใช้ในการคำนวณผลรวมของ **a** และ **b** และแสดงผลในหน้า JSP

2.2 Expression

ประโยชน์: ใช้สำหรับแสดงค่าผลลัพธ์จากการประมวลผลในหน้า JSP โดยไม่ต้องใช้คำสั่ง **out.println()**

ตัวอย่าง:

```
<html>
  <body>
    <h2>The sum is: <%= 5 + 10 %></h2>
  </body>
</html>
```

- ใช้ **expression** เพื่อแสดงผลลัพธ์ของ **5 + 10** บนหน้าเว็บ

2.3 Declaration

ประโยชน์: ใช้ในการประกาศตัวแปรหรือเมธอดที่สามารถใช้ในหน้า JSP ได้

ตัวอย่าง:

```
<%!
  public String greet(String name) {
    return "Hello, " + name + "!";
  }
%>
<html>
  <body>
```

```
<h2><%= greet("John") %></h2>
</body>
</html>
```

- **Declaration** ใช้ในการประกาศเมธอด **greet** และเรียกใช้งานภายในหน้า JSP

3. Action Elements

ประโยชน์: ใช้ในการส่งคำสั่งให้กับ JSP engine เพื่อทำงานบางอย่าง เช่น การเปลี่ยนหน้า, ส่งข้อมูลไปยัง Servlet, หรือใช้ tag library

3.1 [jsp:include](#)

ประโยชน์: ใช้ในการรวมเนื้อหาจากไฟล์อื่นไปยังหน้า JSP (คล้ายกับ **include directive** แต่ใช้ในรูปแบบของ action element)

ตัวอย่าง:

```
<jsp:include page="footer.jsp" />
```

- คำสั่งนี้จะรวมเนื้อหาจาก **footer.jsp** เข้ามาในหน้า JSP ที่ใช้งาน

3.2 [jsp:forward](#)

ประโยชน์: ใช้ในการเปลี่ยนเส้นทาง (redirect) ไปยังหน้าอื่นโดยที่ไม่ต้องส่งคำตอบกลับไปยังลูกค้าก่อน

ตัวอย่าง:

```
<jsp:forward page="nextPage.jsp" />
```

- คำสั่งนี้จะเปลี่ยนเส้นทางการทำงานไปยัง **nextPage.jsp**

3.3 [jsp:param](#)

ประโยชน์: ใช้ในการส่งพารามิเตอร์ไปยัง [jsp:include](#) หรือ [jsp:forward](#)

ตัวอย่าง:

```
<jsp:forward page="nextPage.jsp">
```

```
<jsp:param name="userName" value="John" />
```

```
</jsp:forward>
```

- คำสั่งนี้จะส่งพารามิเตอร์ **userName** ไปยัง **nextPage.jsp** พร้อมค่าที่เป็น **John**

4. Error Handling Elements

ประโยชน์: ใช้ในการจัดการกับข้อผิดพลาดในหน้า JSP

4.1

ประโยชน์: ใช้ในการกำหนดหน้าเว็บที่จะแสดงเมื่อเกิดข้อผิดพลาดขึ้นในแอปพลิเคชัน

ตัวอย่าง:

```
<error-page>
  <exception-type>java.lang.Exception</exception-type>
  <location>/error.jsp</location>
</error-page>
```

- หากเกิดข้อผิดพลาด **Exception** จะส่งไปที่ **error.jsp**

สรุป

Command List ของ JSP รวมถึงคำสั่งต่างๆ ที่ใช้ในการตั้งค่าการทำงานใน JSP โดยสามารถแบ่งได้เป็นหลายประเภท เช่น:

- **Directives:** ใช้ในการกำหนดค่าทั่วไปของหน้า JSP
- **Scripting Elements:** ใช้ในการเขียนโค้ด Java ในหน้า JSP
- **Action Elements:** ใช้ในการส่งคำสั่งพิเศษเช่นการรวมหน้า หรือการเปลี่ยนเส้นทาง
- **Error Handling:** ใช้ในการจัดการข้อผิดพลาดที่เกิดขึ้นในหน้า JSP

ทุกคำสั่งมีประโยชน์และสามารถช่วยเพิ่มความยืดหยุ่นในการพัฒนาเว็บแอปพลิเคชันด้วย JSP

ตัวอย่างเพิ่มเติม ของคำสั่งต่าง ๆ ใน JSP

ต่อไปนี้เป็น ตัวอย่างเพิ่มเติม ของคำสั่งต่างๆ ใน **JSP** ที่สามารถใช้ในการพัฒนาแอปพลิเคชัน โดยทุกคำสั่งมีประโยชน์ในการจัดการเนื้อหาและการทำงานของหน้า JSP:

1. [jsp:useBean](#)

ประโยชน์: ใช้สำหรับการสร้างหรือเข้าถึง **JavaBean** ที่กำหนดไว้ เพื่อใช้งานในหน้า JSP

ตัวอย่าง:

```
<jsp:useBean id="person" class="com.example.Person" scope="session" />
<html>
  <body>
    <h2>Name: <jsp:getProperty name="person" property="name" /></h2>
  </body>
</html>
```

- คำสั่งนี้จะสร้าง **JavaBean** ที่ชื่อว่า **person** และแสดงค่า **name** จาก Bean ในหน้าเว็บ

2. [jsp:setProperty](#)

ประโยชน์: ใช้ในการตั้งค่าคุณสมบัติของ **JavaBean** ที่กำหนดไว้

ตัวอย่าง:

```
<jsp:useBean id="person" class="com.example.Person" scope="session" />
<jsp:setProperty name="person" property="name" value="John Doe" />
```

```
<html>
  <body>
    <h2>Name: <jsp:getProperty name="person" property="name" /></h2>
  </body>
</html>
```

- ใช้ในการตั้งค่าคุณสมบัติ **name** ของ **JavaBean person** เป็น **John Doe**

3. [jsp:getProperty](#)

ประโยชน์: ใช้ในการดึงค่าของคุณสมบัติจาก **JavaBean** ที่กำหนดไว้

ตัวอย่าง:

```
<jsp:useBean id="person" class="com.example.Person" />
<jsp:getProperty name="person" property="name" />
```

- ใช้ในการดึงข้อมูลจาก **JavaBean** เช่น **name** และแสดงในหน้า JSP

4. **<c:out>** (JSTL)

ประโยชน์: ใช้ในการแสดงค่าผลลัพธ์ที่มีการประมวลผลหรือไม่ต้องการ escape HTML (ป้องกัน Cross-site scripting)

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <body>
    <c:out value="${userInput}" />
  </body>
</html>
```

- คำสั่ง **<c:out>** จะแสดงผลค่าที่อยู่ใน **userInput** พร้อมการ escape อัตโนมัติ

5. **<c:if>** (JSTL)

ประโยชน์: ใช้ในการตรวจสอบเงื่อนไขและแสดงผลลัพธ์หากเงื่อนไขเป็นจริง

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <body>
    <c:if test="${userLoggedIn}">
      <p>Welcome back, user!</p>
    </c:if>
  </body>
</html>
```

- **<c:if>** ตรวจสอบว่า **userLoggedIn** เป็นจริงหรือไม่ ถ้าเป็นจริงจะถูกแสดงข้อความทักทาย

6. <c:choose>, <c:when>, <c:otherwise> (JSTL)

ประโยชน์: ใช้ในการจัดการเงื่อนไขหลายๆ อันอย่างมีประสิทธิภาพ

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <body>
    <c:choose>
      <c:when test="${userRole == 'admin'}">
        <p>Welcome, admin!</p>
      </c:when>
      <c:when test="${userRole == 'user'}">
        <p>Welcome, user!</p>
      </c:when>
      <c:otherwise>
        <p>Welcome, guest!</p>
      </c:otherwise>
    </c:choose>
  </body>
</html>
```

- **<c:choose>** ใช้ตรวจสอบเงื่อนไขหลายๆ อย่าง เช่น แสดงข้อความต่างๆ ตามบทบาทผู้ใช้

7. <c:forEach> (JSTL)

ประโยชน์: ใช้ในการวนลูปแสดงรายการข้อมูลที่เป็นคอลเล็กชัน

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <body>
    <ul>
      <c:forEach var="item" items="${itemList}">
        <li>${item}</li>
      </c:forEach>
    </ul>
  </body>
</html>
```

- **<c:forEach>** ใช้ในการวนลูปแสดงรายการ **itemList**

8. [jsp:plugin](#)

ประโยชน์: ใช้ในการสร้าง **Java applet** หรือ **plugin** ในหน้า JSP

ตัวอย่าง:

```
<jsp:plugin type="applet" code="com.example.MyApplet.class" width="300" height="300">
  <param name="param1" value="value1" />
</jsp:plugin>
```

- ใช้ในการฝัง **Java applet** ในหน้าเว็บ

9. [jsp:body](#)

ประโยชน์: ใช้ในการสร้างส่วนที่สามารถแทนที่ได้ใน **custom tag** เมื่อมีการใช้งาน **tag handler**

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <body>
    <c:forEach var="item" items="${items}">
      <jsp:body>
        <p>${item}</p>
      </jsp:body>
    </c:forEach>
  </body>
</html>
```

- ใช้ในการจัดการเนื้อหาภายในส่วนของแท็กที่สามารถแทนที่ได้

10. [jsp:throw](#)

ประโยชน์: ใช้ในการโยน **exception** เพื่อจัดการข้อผิดพลาดในหน้า JSP

ตัวอย่าง:

```
<%@ page errorPage="error.jsp" %>
<jsp:throw exception="java.lang.Exception" />
```

- ใช้ในการโยน **exception** เพื่อให้ **errorPage** รับผิดชอบการจัดการข้อผิดพลาด

สรุป

คำสั่งต่างๆ ที่ใช้ใน JSP นั้นมีหลากหลายประเภทและใช้เพื่อการจัดการข้อมูลในหน้า JSP เช่น:

- การจัดการกับ **JavaBeans** เช่น [jsp:useBean](#), [jsp:setProperty](#), [jsp:getProperty](#)
- การแสดงข้อมูลและตรวจสอบเงื่อนไขต่างๆ ด้วย **JSTL** เช่น **<c:out>**, **<c:forEach>**, **<c:if>**
- การสร้าง **plugin** หรือ **applet** เช่น [jsp:plugin](#)

- การจัดการข้อผิดพลาดด้วย [jsp:throw](#)

ทุกคำสั่งมีบทบาทในการทำให้การพัฒนาเว็บด้วย JSP ง่ายขึ้นและยืดหยุ่นมากขึ้น

คำสั่ง JSP ที่สามารถใช้งานใน NetBeans IDE รันบน Glassfish Web Server

ต่อไปนี้เป็นตัวอย่าง คำสั่ง JSP ที่สามารถใช้งานใน NetBeans IDE รันบน Glassfish Web Server อีก 10 ตัวอย่างที่สามารถนำไปใช้ในโครงการของคุณ:

1. [jsp:forward](#)

ประโยชน์: ใช้ในการเปลี่ยนเส้นทาง (forward) ไปยังหน้าต่างๆ โดยไม่ต้องส่งคำตอบกลับไปยังผู้ใช้ก่อน

ตัวอย่าง:

```
<jsp:forward page="nextPage.jsp" />
```

- คำสั่งนี้จะเปลี่ยนเส้นทางไปยังหน้า **nextPage.jsp**

2. [jsp:include](#)

ประโยชน์: ใช้ในการรวมเนื้อหาของหน้า JSP อื่นเข้ามาในหน้า JSP นี้

ตัวอย่าง:

```
<jsp:include page="header.jsp" />
```

- คำสั่งนี้จะรวมเนื้อหาของ **header.jsp** เข้ามาในหน้า JSP หลัก

3. [jsp:useBean](#)

ประโยชน์: ใช้ในการสร้างและเข้าถึง **JavaBean** เพื่อให้สามารถใช้งานในหน้า JSP ได้

ตัวอย่าง:

```
<jsp:useBean id="user" class="com.example.User" scope="session" />
```

```
<jsp:setProperty name="user" property="username" value="JohnDoe" />
```

```
<p>User name: <jsp:getProperty name="user" property="username" /></p>
```

- คำสั่งนี้จะสร้าง **JavaBean** ที่ชื่อ **user** และตั้งค่า **username** เป็น "JohnDoe"

4. [jsp:setProperty](#)

ประโยชน์: ใช้ในการตั้งค่าคุณสมบัติของ **JavaBean**

ตัวอย่าง:

```
<jsp:useBean id="person" class="com.example.Person" scope="session" />
```

```
<jsp:setProperty name="person" property="name" value="Alice" />
```

```
<p>Person's Name: <jsp:getProperty name="person" property="name" /></p>
```

- คำสั่งนี้จะตั้งค่าคุณสมบัติ **name** ของ **person** เป็น "Alice"

5. [jsp:getProperty](#)

ประโยชน์: ใช้ในการดึงค่าคุณสมบัติจาก **JavaBean**

ตัวอย่าง:

```
<jsp:useBean id="product" class="com.example.Product" />
<p>Product Name: <jsp:getProperty name="product" property="name" /></p>
```

- คำสั่งนี้จะดึงค่าของ **name** จาก **JavaBean** ที่ชื่อว่า **product**

6. jsp:plugin

ประโยชน์: ใช้ในการฝัง **Java applet** หรือ **plugin** ในหน้า JSP

ตัวอย่าง:

```
<jsp:plugin type="applet" code="com.example.MyApplet.class" width="300" height="300">
  <param name="param1" value="value1" />
</jsp:plugin>
```

- ใช้ฝัง **Java applet** ลงในหน้า JSP

7. **<c:out>** (JSTL)

ประโยชน์: ใช้ในการแสดงค่าผลลัพธ์จากการประมวลผลหรือค่าจากตัวแปรต่างๆ

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<p>Welcome, <c:out value="${userName}" />!</p>
```

- คำสั่ง **<c:out>** ใช้ในการแสดงผลค่าของ **userName**

8. **<c:forEach>** (JSTL)

ประโยชน์: ใช้ในการวนลูปแสดงรายการข้อมูลจาก **List** หรือ **Array**

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<ul>
  <c:forEach var="item" items="${itemList}">
    <li>${item}</li>
  </c:forEach>
</ul>
```

- คำสั่ง **<c:forEach>** จะวนลูปรายการใน **itemList** และแสดงในรูปแบบของรายการ

9. **<c:if>** (JSTL)

ประโยชน์: ใช้ในการแสดงเนื้อหาขึ้นอยู่กับเงื่อนไขที่กำหนด

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<c:if test="${userLoggedIn}">
```

```
<p>Welcome back, user!</p>
```

```
</c:if>
```

- คำสั่ง **<c:if>** ใช้ในการตรวจสอบเงื่อนไข **userLoggedIn** และแสดงข้อความต้อนรับหากเงื่อนไขเป็นจริง

10. [jsp:errorPage](#)

ประโยชน์: ใช้ในการกำหนดหน้าแสดงข้อผิดพลาดเมื่อเกิด **exception** ขึ้นในหน้า JSP

ตัวอย่าง:

```
<%@ page errorPage="error.jsp" %>
```

```
<html>
```

```
<body>
```

```
<h2>This is a sample JSP page</h2>
```

```
</body>
```

```
</html>
```

- คำสั่ง **<%@ page errorPage="error.jsp" %>** กำหนดว่าเมื่อเกิดข้อผิดพลาดในหน้า JSP จะส่งไปที่ **error.jsp**

สรุป

คำสั่ง JSP ในตัวอย่างที่ให้มาทั้งหมดสามารถใช้งานได้ทั้งใน **NetBeans IDE** รันบน **Glassfish Web Server** เพื่อทำงานต่างๆ เช่น:

- การรวมหรือเปลี่ยนเส้นทางไปยังหน้าต่างๆ
- การใช้ **JavaBeans** เพื่อจัดการข้อมูล
- การแสดงผลหรือจัดการเงื่อนไขด้วย **JSTL**
- การฝัง **Java applet** หรือ **plugin** ในหน้า JSP
- การจัดการข้อผิดพลาดด้วยการกำหนดหน้า **errorPage**

ทุกคำสั่งเหล่านี้ช่วยให้การพัฒนาแอปพลิเคชันบน **Glassfish Web Server** ทำได้สะดวกและมีประสิทธิภาพ

ใน **JSP** (JavaServer Pages) มีหลายคำสั่งที่ใช้ในการแสดงผลข้อมูลบนหน้าเว็บ ต่อไปนี้คือลิสต์คำสั่งหลักที่ใช้ในการแสดงผลใน JSP:

1. ใช้การแสดงผลด้วย **out.print()**

ประโยชน์: ใช้เพื่อแสดงข้อความหรือข้อมูลออกมาบนหน้าเว็บ

ตัวอย่าง:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"%>
```

```
<html>
  <body>
    <%
      out.print("Hello, JSP!");
    %>
  </body>
</html>
```

- **out.print()** ใช้แสดงข้อความ "Hello, JSP!" ในหน้าเว็บ

2. <%= expression %>

ประโยชน์: ใช้ในการแสดงผลค่าของ **Java expression** โดยตรงใน HTML

ตัวอย่าง:

```
<html>
  <body>
    <h1>Welcome, <%= "John" %>!</h1>
  </body>
</html>
```

- **<%= expression %>** จะแสดงข้อความ "John" ในหน้าเว็บ

3. [jsp:out](#)

ประโยชน์: ใช้ในการแสดงผลข้อมูลผ่านการประมวลผลจาก **JavaBean** หรือ **JSTL** ที่มีการ escape HTML

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <body>
    <c:out value="${userName}" />
  </body>
</html>
```

- **<c:out>** ใช้แสดงค่าของ **userName** ซึ่งจะทำให้การ escape ตัวอักษรพิเศษ เช่น < หรือ > ให้เป็นรหัส HTML

4. [jsp:getProperty](#)

ประโยชน์: ใช้ในการดึงข้อมูลจาก **JavaBean** และแสดงในหน้า JSP

ตัวอย่าง:

```
<jsp:useBean id="person" class="com.example.Person" />
<p>Person's Name: <jsp:getProperty name="person" property="name" /></p>
```

- **<jsp:getProperty>** ใช้ในการดึงค่าของ **name** จาก **JavaBean person**

5. **<%= request.getParameter() %>**

ประโยชน์: ใช้ในการแสดงค่าของ **parameter** ที่รับมาจาก URL หรือฟอร์ม

ตัวอย่าง:

```
<html>
  <body>
    <h1>Welcome, <%= request.getParameter("username") %>!/h1>
  </body>
</html>
```

- **request.getParameter("username")** จะดึงค่าของ **username** จาก **query string** หรือ **form** และแสดงในหน้าเว็บ

6. **<c:forEach> (JSTL)**

ประโยชน์: ใช้ในการวนลูปแสดงรายการข้อมูลจาก **List** หรือ **Array**

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <body>
    <ul>
      <c:forEach var="item" items="${itemList}">
        <li>${item}</li>
      </c:forEach>
    </ul>
  </body>
</html>
```

- **<c:forEach>** ใช้ในการวนลูปรายการ **itemList** และแสดงแต่ละรายการในรูปแบบของรายการ

7. **<c:out> (JSTL)**

ประโยชน์: ใช้ในการแสดงค่าผลลัพธ์จากการประมวลผลหรือค่าจากตัวแปรต่างๆ

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<p>Hello, <c:out value="${userName}" />!/p>
```

- **<c:out>** ใช้แสดงค่าของ **userName** และทำการ escape HTML อัตโนมัติ

8. **[jsp:forward](#)**

ประโยชน์: ใช้ในการเปลี่ยนเส้นทางไปยังหน้า JSP อื่นโดยตรง

ตัวอย่าง:

```
<jsp:forward page="nextPage.jsp" />
```

- **<jsp:forward>** ใช้ในการเปลี่ยนเส้นทางไปยังหน้า **nextPage.jsp**

9. **<jsp:include>**

ประโยชน์: ใช้ในการรวมเนื้อหาของหน้า JSP อื่น

ตัวอย่าง:

```
<jsp:include page="header.jsp" />
```

- **<jsp:include>** จะรวมเนื้อหาจากหน้า **header.jsp** เข้ามาในหน้า JSP หลัก

10. **<c:if>** (JSTL)

ประโยชน์: ใช้ในการแสดงเนื้อหาขึ้นอยู่กับเงื่อนไขที่กำหนด

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
```

```
<c:if test="${userLoggedIn}">
```

```
    <p>Welcome back, user!</p>
```

```
</c:if>
```

- **<c:if>** ใช้ในการแสดงข้อความต้อนรับเมื่อ **userLoggedIn** เป็นจริง

สรุป

คำสั่งที่ใช้ในการ **แสดงผล** ใน **JSP** มีหลายรูปแบบที่เหมาะสมกับสถานการณ์ต่างๆ:

1. **out.print()** และ **<%= %>** สำหรับการแสดงผลข้อความที่ง่าย
2. **<jsp:getProperty>** สำหรับดึงข้อมูลจาก **JavaBean**
3. **<c:out>** สำหรับการแสดงข้อมูลจาก **JSTL** และ escape HTML
4. **<c:forEach>** และ **<c:if>** สำหรับการแสดงข้อมูลตามเงื่อนไขหรือการวนลูป
5. **<jsp:include>** และ **<jsp:forward>** สำหรับการรวมหรือเปลี่ยนเส้นทางไปยังหน้าอื่น

คำสั่งเหล่านี้ทำให้การพัฒนาเว็บไซต์ด้วย JSP มีความยืดหยุ่นและง่ายต่อการจัดการข้อมูล

JSTL (JavaServer Pages Standard Tag Library)

ใน **JSP** (JavaServer Pages) ยังมีคำสั่งและฟังก์ชันอื่นๆ ที่สามารถใช้ในการแสดงผลข้อมูลได้ รวมถึงคำสั่งที่ใช้ในการจัดการกับฟอร์ม, การแสดงผลจากฐานข้อมูล, และการใช้งาน **JSTL (JavaServer Pages Standard Tag Library)** อย่างมีประสิทธิภาพ นี่คือนิยามคำสั่งเพิ่มเติมที่สามารถใช้ในการแสดงผลใน JSP:

```
11. <%= new java.util.Date() %>
```

ประโยชน์: ใช้แสดงผลวันและเวลาปัจจุบัน

ตัวอย่าง:

```
<html>
  <body>
    <p>Current date and time: <%= new java.util.Date() %></p>
  </body>
</html>
```

- **new java.util.Date()** จะแสดงวันที่และเวลาปัจจุบันในรูปแบบของ **Date**

12. [jsp:plugin](#)

ประโยชน์: ใช้ในการฝัง **Java applet** หรือ **Java plugin** ลงในหน้า JSP

ตัวอย่าง:

```
<jsp:plugin type="applet" code="com.example.MyApplet.class" width="200" height="200">
  <param name="param1" value="value1" />
</jsp:plugin>
```

- **<jsp:plugin>** ใช้ในการฝัง **Java applet** ลงในหน้า

13. **<%= request.getAttribute() %>**

ประโยชน์: ใช้ในการดึงค่า **attribute** ที่ส่งมาจาก **Servlet** และแสดงผลในหน้า JSP

ตัวอย่าง:

```
<html>
  <body>
    <h1>Attribute: <%= request.getAttribute("user") %></h1>
  </body>
</html>
```

- **request.getAttribute("user")** จะดึงค่าของ **user** ที่ถูกตั้งค่าใน **Servlet** และแสดงในหน้า JSP

14. **<%= session.getAttribute() %>**

ประโยชน์: ใช้ในการดึงค่า **session attribute** ที่ถูกเก็บใน **Session** และแสดงในหน้า JSP

ตัวอย่าง:

```
<html>
  <body>
    <h1>Session User: <%= session.getAttribute("username") %></h1>
  </body>
</html>
```

- **session.getAttribute("username")** จะดึงค่าของ **username** จาก **Session**

15. **<%= pageContext.getRequest().getParameter() %>**

ประโยชน์: ใช้ในการดึงค่า **parameter** จาก **request** โดยตรงจาก **pageContext**

ตัวอย่าง:

```
<html>
  <body>
    <h1>Received Parameter: <%= pageContext.getRequest().getParameter("username")
%></h1>
  </body>
</html>
```

- **pageContext.getRequest().getParameter("username")** จะดึงค่าจาก **request parameter** ชื่อ **username**

16. [jsp:errorPage](#)

ประโยชน์: ใช้ในการกำหนดหน้าแสดงข้อผิดพลาดเมื่อเกิด **exception** ขึ้นในหน้า JSP

ตัวอย่าง:

```
<%@ page errorPage="error.jsp" %>
<html>
  <body>
    <h2>This is a sample JSP page</h2>
  </body>
</html>
```

- **<%@ page errorPage="error.jsp" %>** ใช้กำหนดให้เมื่อมี **exception** จะไปแสดงหน้า **error.jsp**

17. **<%= Math.random() %>**

ประโยชน์: ใช้ในการแสดงค่าตัวเลขสุ่ม

ตัวอย่าง:

```
<html>
  <body>
    <h1>Random Number: <%= Math.random() %></h1>
  </body>
</html>
```

- **Math.random()** จะแสดงค่าตัวเลขสุ่มระหว่าง 0 ถึง 1

18. [jsp:directive](#)

ประโยชน์: ใช้ในการกำหนด **directive** ของ JSP เช่น **page** directive, **taglib** directive

ตัวอย่าง:

```
<%@ page contentType="text/html; charset=UTF-8" language="java" %>
```

```
<html>
  <body>
    <h1>Page Directive Example</h1>
  </body>
</html>
```

- **<%@ page contentType="text/html; charset=UTF-8" %>** กำหนดประเภทของเนื้อหาและ การเข้ารหัสของหน้า JSP

19. **<%= request.getServerName() %>**

ประโยชน์: ใช้ในการแสดงชื่อของเซิร์ฟเวอร์ที่ผู้ใช้เข้าถึง

ตัวอย่าง:

```
<html>
  <body>
    <h1>Server Name: <%= request.getServerName() %></h1>
  </body>
</html>
```

- **request.getServerName()** จะแสดงชื่อเซิร์ฟเวอร์

20. **<c:choose>, <c:when>, <c:otherwise> (JSTL)**

ประโยชน์: ใช้ในการเลือกแสดงเนื้อหาตามเงื่อนไขแบบ **switch-case**

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
  <body>
    <c:choose>
      <c:when test="{userStatus == 'active'}">
        <p>Welcome back, active user!</p>
      </c:when>
      <c:otherwise>
        <p>Your account is inactive.</p>
      </c:otherwise>
    </c:choose>
  </body>
</html>
```

- **<c:choose>** ใช้ในการเลือกเงื่อนไขแบบ **switch-case** โดยการเลือกแสดงข้อความตามสถานะ **userStatus**

สรุป

คำสั่งในการ **แสดงผล** ที่สามารถใช้ใน **JSP** นอกจากที่กล่าวไปก่อนหน้านี้ยังมีตัวเลือกมากมาย เช่น:

1. การใช้ **Math.random()** เพื่อแสดงตัวเลขสุ่ม
2. การดึงค่าจาก **request** และ **session**
3. การแสดงผลวันและเวลา
4. การใช้ **JSTL** เพื่อทำงานกับเงื่อนไขและการวนลูป
5. การใช้ **[jsp:plugin](#)** สำหรับการฝัง **Java applet** ในหน้า JSP

ทุกคำสั่งนี้ช่วยให้การพัฒนาแอปพลิเคชัน JSP มีความยืดหยุ่นมากขึ้นและสามารถตอบสนองความต้องการของผู้พัฒนาได้หลากหลาย

JavaBeans, JSTL, EL (Expression Language) และ การจัดการกับข้อผิดพลาด

ใน **JSP** (JavaServer Pages) นอกจากคำสั่งที่กล่าวไปแล้ว ยังมีคำสั่งอื่นๆ ที่ใช้ในการแสดงผลข้อมูล และจัดการกับข้อมูลต่างๆ รวมถึงการใช้งานกับ **JavaBeans**, **JSTL**, **EL (Expression Language)** และ การจัดการกับข้อผิดพลาด ซึ่งสามารถใช้ในการพัฒนาแอปพลิเคชัน JSP ได้อย่างมีประสิทธิภาพ นี่คือการเพิ่มเติมนำที่สามารถใช้ในการแสดงผลใน JSP:

21. `<%= new java.text.SimpleDateFormat("yyyy-MM-dd").format(new java.util.Date()) %>`

ประโยชน์: ใช้ในการแสดงวันที่และเวลาปัจจุบันในรูปแบบที่กำหนด

ตัวอย่าง:

```
<html>
  <body>
    <h1>Current Date: <%= new java.text.SimpleDateFormat("yyyy-MM-dd").format(new
java.util.Date()) %></h1>
  </body>
</html>
```

- **`new java.text.SimpleDateFormat("yyyy-MM-dd")`** ใช้ในการแสดงวันที่ในรูปแบบปี-เดือน-วัน

22. `<%= request.getRequestURI() %>`

ประโยชน์: ใช้ในการแสดง URI (Uniform Resource Identifier) ของหน้าเว็บที่ถูกเรียก

ตัวอย่าง:

```
<html>
  <body>
    <h1>Current URI: <%= request.getRequestURI() %></h1>
```

```
</body>
```

```
</html>
```

- **request.getRequestURI()** จะแสดง URI ของหน้าที่กำลังถูกเข้าถึง

23. `<%= response.getStatus() %>`

ประโยชน์: ใช้ในการแสดงสถานะของ HTTP response ที่ส่งกลับไปยังผู้ใช้

ตัวอย่าง:

```
<html>
```

```
<body>
```

```
<h1>Response Status: <%= response.getStatus() %></h1>
```

```
</body>
```

```
</html>
```

- **response.getStatus()** จะแสดงรหัสสถานะของ HTTP response (เช่น 200 สำหรับ OK)

24. [jsp:param](#)

ประโยชน์: ใช้ในการส่งพารามิเตอร์ไปยังหน้าหรือแอปพลิเคชันอื่น

ตัวอย่าง:

```
<jsp:forward page="nextPage.jsp">
```

```
<jsp:param name="username" value="john_doe" />
```

```
</jsp:forward>
```

- **<jsp:param>** ส่งพารามิเตอร์ **username** ไปยังหน้า **nextPage.jsp**

25. [jsp:attribute](#)

ประโยชน์: ใช้ในการกำหนดค่าให้กับ **tag** ที่ถูกกำหนดใน **custom tags** หรือ **JSP fragments**

ตัวอย่าง:

```
<jsp:root xmlns:jsp="http://java.sun.com/JSP/Page">
```

```
<jsp:attribute name="class">example-class</jsp:attribute>
```

```
</jsp:root>
```

- **<jsp:attribute>** ใช้กำหนดค่า **class** สำหรับ **custom tag**

26. `<%= Math.max(10, 20) %>`

ประโยชน์: ใช้ในการคำนวณหรือหาค่าสูงสุดของตัวเลข

ตัวอย่าง:

```
<html>
```

```
<body>
```

```
<h1>Maximum Value: <%= Math.max(10, 20) %></h1>
```

```
</body>
```

```
</html>
```

- **Math.max(10, 20)** จะคำนวณและแสดงค่ามากที่สุดระหว่าง 10 และ 20

27. [jsp:body](#)

ประโยชน์: ใช้ใน **custom tags** เพื่อให้สามารถจัดการกับเนื้อหาภายในของ tag นั้นๆ

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<%@ taglib uri="http://java.sun.com/jsp/jstl/functions" prefix="fn" %>
<jsp:body>
    <h1>Body content goes here!</h1>
</jsp:body>
```

- **<jsp:body>** ใช้ภายใน **custom tag** เพื่อให้สามารถจัดการเนื้อหาภายใน tag ได้

28. [jsp:invoke](#)

ประโยชน์: ใช้ในการเรียกใช้ **tag handler** ในการประมวลผลข้อมูลภายใน **custom tag**

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<jsp:invoke>
    <h1>Invoke tag content</h1>
</jsp:invoke>
```

- **<jsp:invoke>** ใช้ในการประมวลผลเนื้อหาภายใน **custom tag**

29. [jsp:include](#) กับ Request Dispatcher

ประโยชน์: ใช้ในการรวมไฟล์ JSP อื่นเข้าไปในหน้า JSP หลัก ผ่าน **RequestDispatcher**

ตัวอย่าง:

```
<%@ page language="java" contentType="text/html; charset=ISO-8859-1"%>
<jsp:include page="footer.jsp" />
```

- **<jsp:include>** ใช้ในการรวม **footer.jsp** ในหน้าหลัก

30. **<c:choose>**, **<c:when>**, **<c:otherwise>** (JSTL)

ประโยชน์: ใช้ในการเลือกแสดงเนื้อหาตามเงื่อนไขแบบ **switch-case**

ตัวอย่าง:

```
<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
    <body>
        <c:choose>
            <c:when test="${userRole == 'admin'}">
                <p>Welcome, Admin!</p>
            </c:when>
```

```

        <c:otherwise>
            <p>Welcome, User!</p>
        </c:otherwise>
    </c:choose>
</body>
</html>

```

- **<c:choose>** ใช้ในกรณีที่ต้องการเลือกการแสดงผลข้อความตามเงื่อนไขต่างๆ เช่น **userRole**

สรุป

คำสั่งในการ **แสดงผล** ใน **JSP** มีหลายรูปแบบและหลากหลายวิธี เช่น:

- การใช้ **Java methods** เช่น `Math.max()`, `new java.util.Date()`, หรือ `request.getRequestURI()`
- การใช้ **JSP standard tags** เช่น [jsp:param](#), [jsp:include](#), [jsp:forward](#)
- การใช้งาน **JSTL** เช่น **<c:choose>**, **<c:forEach>** และ **<c:out>**
- การใช้ **Expression Language (EL)** ในการดึงค่าตัวแปรและแสดงผล

การใช้คำสั่งเหล่านี้ช่วยให้การพัฒนาเว็บแอปพลิเคชันมีความยืดหยุ่นและสามารถทำงานได้ดีขึ้นในหลากหลายสถานการณ์

คำสั่งแสดงผลใน **JSP** ยังมีหลายตัวเลือกที่สามารถใช้งานได้เพื่อเพิ่มความยืดหยุ่นในการพัฒนาเว็บแอปพลิเคชัน นี่คือนิยามเพิ่มเติมของคำสั่งแสดงผลใน JSP:

31. <%= request.getRemoteAddr() %>

ประโยชน์: ใช้ในการดึงที่อยู่ IP ของผู้ใช้ที่เข้าถึงเว็บแอปพลิเคชัน

ตัวอย่าง:

```

<html>
    <body>
        <h1>Your IP Address: <%= request.getRemoteAddr() %></h1>
    </body>
</html>

```

- **request.getRemoteAddr()** จะแสดงที่อยู่ IP ของผู้ใช้

32. <%= session.getId() %>

ประโยชน์: ใช้ในการแสดง **Session ID** ของผู้ใช้

ตัวอย่าง:

```

<html>
    <body>

```

```

    <h1>Session ID: <%= session.getId() %></h1>
</body>
</html>

```

- **session.getId()** จะดึงค่า **Session ID** ของผู้ใช้ปัจจุบัน

33. <%= request.getHeader("User-Agent") %>

ประโยชน์: ใช้ในการดึงข้อมูลเกี่ยวกับเบราว์เซอร์และระบบปฏิบัติการของผู้ใช้
ตัวอย่าง:

```

<html>
<body>
    <h1>User-Agent: <%= request.getHeader("User-Agent") %></h1>
</body>
</html>

```

- **request.getHeader("User-Agent")** จะแสดงข้อมูลเกี่ยวกับเบราว์เซอร์และระบบปฏิบัติการที่ผู้ใช้ใช้งาน

34. <%= pageContext.getResponse().getLocale() %>

ประโยชน์: ใช้ในการดึงภาษาหรือภูมิภาคของผู้ใช้จาก HTTP response
ตัวอย่าง:

```

<html>
<body>
    <h1>User Locale: <%= pageContext.getResponse().getLocale() %></h1>
</body>
</html>

```

- **pageContext.getResponse().getLocale()** จะแสดง **locale** ของผู้ใช้ เช่น ภาษาและประเทศที่ใช้งาน

35. <%= request.getParameterValues("paramName") %>

ประโยชน์: ใช้ในการดึงค่าหลายๆ ค่า **parameter** จาก **request** ที่มีชื่อเดียวกัน
ตัวอย่าง:

```

<html>
<body>
    <h1>Selected Values:</h1>
    <ul>
        <%
            String[] values = request.getParameterValues("category");
            if (values != null) {

```

```

        for (String value : values) {
            out.println("<li>" + value + "</li>");
        }
    }
    %>
</ul>
</body>
</html>

```

- **request.getParameterValues("category")** จะดึงค่าทั้งหมดที่ส่งมาใน **parameter** ชื่อ **category**

36. <%= Math.random() * 100 %>

ประโยชน์: ใช้ในการแสดงตัวเลขสุ่มที่อยู่ในช่วง 0-100

ตัวอย่าง:

```

<html>
<body>
    <h1>Random Number: <%= Math.random() * 100 %></h1>
</body>
</html>

```

- **Math.random() * 100** จะแสดงค่าตัวเลขสุ่มระหว่าง 0 ถึง 100

37. <%= request.getAttribute("attributeName") %>

ประโยชน์: ใช้ในการดึงค่าของ **attribute** ที่ถูกเก็บใน **request** scope

ตัวอย่าง:

```

<html>
<body>
    <h1>Request Attribute: <%= request.getAttribute("userName") %></h1>
</body>
</html>

```

- **request.getAttribute("userName")** จะดึงค่าของ **userName** ที่ถูกตั้งใน **request** scope

38. <c:out value="\${user}" />

ประโยชน์: ใช้ในการแสดงค่าของตัวแปรใน **JSTL** โดยการใช้งาน **EL (Expression Language)**

ตัวอย่าง:

```

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
<body>

```

```

    <h1>User: <c:out value="\${user}" /></h1>
</body>
</html>

```

- **<c:out>** ใช้ในการแสดงค่าของตัวแปร **user** ที่เก็บใน **EL** (Expression Language)

39. <%= pageContext.getRequest().getMethod() %>

ประโยชน์: ใช้ในการดึง **HTTP method** ที่ถูกใช้ใน request (GET, POST, etc.)

ตัวอย่าง:

```

<html>
<body>
    <h1>HTTP Method: <%= pageContext.getRequest().getMethod() %></h1>
</body>
</html>

```

- **pageContext.getRequest().getMethod()** จะแสดงประเภทของ **HTTP method** เช่น **GET** หรือ **POST**

40. <c:forEach var="item" items="\\${itemsList}">

ประโยชน์: ใช้ในการวนลูปแสดงข้อมูลจาก **Collection** หรือ **Array** ใน **JSTL**

ตัวอย่าง:

```

<%@ taglib uri="http://java.sun.com/jsp/jstl/core" prefix="c" %>
<html>
<body>
    <h1>Item List:</h1>
    <ul>
        <c:forEach var="item" items="\${itemsList}">
            <li>\${item}</li>
        </c:forEach>
    </ul>
</body>
</html>

```

- **<c:forEach>** ใช้ในการวนลูปผ่าน **itemsList** และแสดงผลแต่ละ **item**

สรุป

คำสั่งแสดงผลใน **JSP** ที่สามารถใช้ได้มีมากมายเช่น:

- การดึงข้อมูลเกี่ยวกับ **IP Address**, **Session ID**, **User-Agent**
- การใช้งาน **JSTL** เพื่อทำงานกับข้อมูลแบบ **forEach**, **choose**, **out**

- การคำนวณค่าหรือแสดง **random numbers**
- การจัดการกับ **request attributes, parameters** และ **headers**

ทั้งหมดนี้ช่วยให้สามารถแสดงผลข้อมูลและทำงานกับข้อมูลในเว็บแอปพลิเคชัน JSP ได้อย่างยืดหยุ่นและมีประสิทธิภาพ

คำสั่งรับข้อมูลใน JSP (JSP Input Handling) และรายละเอียดแต่ละประเภท

JSP สามารถรับข้อมูลจากผู้ใช้งานผ่าน **HTML Forms**, **URL Parameters**, และ **Cookies** โดยใช้ `request` และอ็อบเจกต์ที่เกี่ยวข้อง เช่น `request.getParameter()`, `session`, และ `application` นอกจากนี้ยังสามารถรับข้อมูลจาก **JSON**, **AJAX**, หรือ **Web APIs** ได้อีกด้วย

1. การรับข้อมูลจาก HTML Forms (Form Data)

JSP สามารถรับข้อมูลจากฟอร์ม HTML ที่ใช้ `method="GET"` หรือ `method="POST"` โดยใช้ `request.getParameter()` หรือ `request.getParameterValues()` ในกรณีที่มีหลายค่าที่มีชื่อเดียวกัน

(1.1) `request.getParameter(String name)`

ใช้รับค่าของฟิลด์เดียวจากฟอร์มที่ส่งมาด้วย **GET** หรือ **POST**

ตัวอย่าง **HTML Form**

```
<form action="process.jsp" method="post">
    Username: <input type="text" name="username">
    <br>
    Password: <input type="password" name="password">
    <br>
    <input type="submit" value="Submit">
</form>
```

ตัวอย่าง **JSP (process.jsp)**

```
<%
    String username = request.getParameter("username");
    String password = request.getParameter("password");
%>
<p>Username: <%= username %></p>
<p>Password: <%= password %></p>
```

(1.2) `request.getParameterValues(String name)`

ใช้รับค่าหลายค่าที่ส่งมาจากฟอร์ม เช่น **Checkboxes**

ตัวอย่าง HTML Form

```
<form action="process.jsp" method="post">
  <p>Select your favorite programming languages:</p>
  <input type="checkbox" name="language" value="Java"> Java
  <input type="checkbox" name="language" value="Python"> Python
  <input type="checkbox" name="language" value="JavaScript"> JavaScript
  <br>
  <input type="submit" value="Submit">
</form>
```

ตัวอย่าง JSP (process.jsp)

```
<%
  String[] languages = request.getParameterValues("language");
  if (languages != null) {
    for (String lang : languages) {
      out.println("<p>Selected: " + lang + "</p>");
    }
  } else {
    out.println("<p>No language selected.</p>");
  }
%>
```

2. การรับข้อมูลจาก URL Parameters

URL Parameters ใช้เมื่อมีการส่งค่าผ่าน **query string** เช่น page.jsp?name=John&age=30

(2.1) การรับค่าจาก Query String

```
<%
  String name = request.getParameter("name");
  String age = request.getParameter("age");
%>
<p>Name: <%= name %></p>
<p>Age: <%= age %></p>
```

- เมื่อเรียก URL: **page.jsp?name=John&age=30**
- Output:
- Name: John

- Age: 30

3. การรับข้อมูลจาก Cookies

Cookies ใช้เก็บข้อมูลของผู้ใช้ เช่น session หรือ preferences

(3.1) การอ่านค่าจาก Cookies

```
<%
```

```
    Cookie[] cookies = request.getCookies();
```

```
    if (cookies != null) {
```

```
        for (Cookie cookie : cookies) {
```

```
            out.println("<p>Cookie Name: " + cookie.getName() + ", Value: " + cookie.getValue() +
```

```
"</p>");
```

```
        }
```

```
    }
```

```
%>
```

(3.2) การสร้างและส่ง Cookies

```
<%
```

```
    Cookie userCookie = new Cookie("username", "JohnDoe");
```

```
    userCookie.setMaxAge(60 * 60 * 24); // 1 day
```

```
    response.addCookie(userCookie);
```

```
%>
```

```
<p>Cookie has been set.</p>
```

4. การรับข้อมูลจาก Session

Session ใช้เก็บข้อมูลของผู้ใช้ขณะที่กำลังใช้งานเว็บไซต์

(4.1) การบันทึกข้อมูลลงใน Session

```
<%
```

```
    session.setAttribute("user", "JohnDoe");
```

```
%>
```

```
<p>Session set successfully.</p>
```

(4.2) การดึงค่าจาก Session

```
<%
```

```
    String user = (String) session.getAttribute("user");
```

```
%>
```

```
<p>Logged in user: <%= user %></p>
```

5. การรับข้อมูลจาก Application Scope

Application Scope ใช้สำหรับเก็บข้อมูลที่ใช้ร่วมกันทั้งเว็บไซต์

(5.1) การกำหนดค่าใน Application Scope

```
<%
    application.setAttribute("appName", "My JSP Web Application");
%>
```

(5.2) การอ่านค่าใน Application Scope

```
<%
    String appName = (String) application.getAttribute("appName");
%>
<p>Application Name: <%= appName %></p>
```

6. การรับข้อมูลจาก Headers ของ HTTP Request

Headers ใช้สำหรับข้อมูลเกี่ยวกับเบราว์เซอร์, session, หรือ authentication

(6.1) การอ่านค่า Headers

```
<%
    String userAgent = request.getHeader("User-Agent");
%>
<p>User-Agent: <%= userAgent %></p>
```

7. การรับข้อมูลจาก JSON (AJAX Call)

JSP สามารถรับข้อมูล JSON ผ่าน request.getReader()

(7.1) JavaScript AJAX Call

```
fetch("process.jsp", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ username: "JohnDoe", age: 30 })
})
.then(response => response.text())
.then(data => console.log(data));
```

(7.2) JSP อ่าน JSON

```
<%@ page import="java.io.*, org.json.JSONObject" %>
<%
```

```

BufferedReader reader = request.getReader();
StringBuilder jsonInput = new StringBuilder();
String line;

while ((line = reader.readLine()) != null) {
    jsonInput.append(line);
}

JSONObject jsonObject = new JSONObject(jsonInput.toString());
String username = jsonObject.getString("username");
int age = jsonObject.getInt("age");

out.println("Received Username: " + username);
out.println("Received Age: " + age);
%>

```

สรุป

วิธีการรับข้อมูล	คำอธิบาย	ตัวอย่างคำสั่ง
Form (GET, POST)	รับข้อมูลจากฟอร์ม HTML	request.getParameter("username")
Checkboxes	รับค่าหลายค่า	request.getParameterValues("language")
URL Parameters	รับค่าจาก query string	request.getParameter("name")
Cookies	รับค่าจาก Cookies	request.getCookies()
Session	รับค่าจาก Session	session.getAttribute("user")
Application Scope	รับค่าระดับแอปพลิเคชัน	application.getAttribute("appName")
HTTP Headers	อ่าน Headers ของ request	request.getHeader("User-Agent")
AJAX JSON	รับข้อมูล JSON จาก client	request.getReader()

วิธีเหล่านี้ช่วยให้ JSP สามารถรับข้อมูลจากผู้ใช้และนำไปใช้ต่อยอดในการพัฒนาเว็บแอปพลิเคชันแบบไดนามิกได้

เพิ่มวิธีการรับข้อมูลใน JSP อย่างละเอียดเพิ่มเติม

นอกจากวิธีพื้นฐานที่ได้กล่าวไปแล้ว ยังมีวิธีอื่น ๆ ที่ JSP สามารถใช้รับข้อมูลได้ เช่น **Multi-part Form Data** (สำหรับการอัปโหลดไฟล์), **Servlet Context Parameters**, **WebSocket**, และ **Hidden Fields** ในฟอร์ม HTML

8. การรับข้อมูลจาก Hidden Fields

Hidden Fields ใช้สำหรับส่งข้อมูลที่ไม่ต้องการให้ผู้ใช้เห็นในฟอร์ม HTML

(8.1) ตัวอย่างการใช้ Hidden Fields

HTML Form (index.jsp)

```
<form action="process.jsp" method="post">
  <input type="hidden" name="sessionID" value="ABC123XYZ">
  <input type="text" name="username" placeholder="Enter your name">
  <input type="submit" value="Submit">
</form>
```

JSP (process.jsp)

```
<%
  String username = request.getParameter("username");
  String sessionID = request.getParameter("sessionID");
%>
<p>Username: <%= username %></p>
<p>Session ID: <%= sessionID %></p>
```

9. การรับข้อมูลจาก Multi-part Form Data (การอัปโหลดไฟล์)

ไฟล์ที่อัปโหลดผ่านฟอร์ม HTML ต้องใช้ไลบรารี Apache Commons FileUpload หรือ Jakarta Servlet API

(9.1) การตั้งค่า HTML Form

```
<form action="upload.jsp" method="post" enctype="multipart/form-data">
  <input type="file" name="file">
  <input type="submit" value="Upload">
</form>
```

(9.2) การรับไฟล์ใน JSP (upload.jsp)

```
<%@ page import="java.io.*, java.util.*, javax.servlet.*, javax.servlet.http.*,
org.apache.commons.fileupload.*, org.apache.commons.fileupload.disk.*,
org.apache.commons.fileupload.servlet.*" %>
```

```

<%
    DiskFileItemFactory factory = new DiskFileItemFactory();
    ServletFileUpload upload = new ServletFileUpload(factory);
    List<FileItem> items = upload.parseRequest(request);

    for (FileItem item : items) {
        if (!item.isFormField()) {
            String fileName = item.getName();
            File file = new File("C:/uploads/" + fileName);
            item.write(file);
            out.println("File uploaded: " + fileName);
        }
    }
%>

```

10. การรับข้อมูลจาก Servlet Context Parameters

สามารถกำหนดค่าคงที่ที่ใช้ในแอปพลิเคชันผ่าน **web.xml** และดึงมาใช้ใน JSP

(10.1) ตั้งค่าใน web.xml

```

<context-param>
    <param-name>appTitle</param-name>
    <param-value>My JSP Application</param-value>
</context-param>

```

(10.2) ดึงค่าจาก Servlet Context (config.jsp)

```

<%
    ServletContext context = getServletContext();
    String appTitle = context.getInitParameter("appTitle");
%>
<p>Application Title: <%= appTitle %></p>

```

11. การรับข้อมูลผ่าน WebSockets (Real-time Communication)

JSP รองรับ WebSocket เพื่อรับส่งข้อมูลแบบ **Real-time**

(11.1) สร้าง WebSocket Server (WebSocketServer.java)

```

import javax.websocket.*;
import javax.websocket.server.ServerEndpoint;

```

```
import java.io.IOException;
```

```
@ServerEndpoint("/chat")
```

```
public class WebSocketServer {
```

```
    @OnMessage
```

```
    public void onMessage(String message, Session session) throws IOException {
```

```
        session.getBasicRemote().sendText("Received: " + message);
```

```
    }
```

```
}
```

(11.2) การเชื่อมต่อจาก JSP (chat.jsp)

```
<script>
```

```
    var ws = new WebSocket("ws://localhost:8080/chat");
```

```
    ws.onopen = function() {
```

```
        ws.send("Hello Server!");
```

```
    };
```

```
    ws.onmessage = function(event) {
```

```
        console.log("Message from server: " + event.data);
```

```
    };
```

```
</script>
```

12. การรับข้อมูลจาก Request Dispatcher (Forward Data)

ใช้ **RequestDispatcher** เพื่อส่งข้อมูลไปยังหน้า JSP อื่น

(12.1) ตั้งค่าข้อมูลใน Servlet

```
RequestDispatcher dispatcher = request.getRequestDispatcher("display.jsp");
```

```
request.setAttribute("username", "JohnDoe");
```

```
dispatcher.forward(request, response);
```

(12.2) รับค่าที่ส่งมาภายใน display.jsp

```
<%
```

```
    String username = (String) request.getAttribute("username");
```

```
%>
```

```
<p>Username from Servlet: <%= username %></p>
```

13. การรับข้อมูลจาก Environment Variables

JSP สามารถอ่านค่าตัวแปรแวดล้อมจากระบบปฏิบัติการได้

(13.1) รับค่าจาก Environment Variables

```
<%
    String javaHome = System.getenv("JAVA_HOME");
%>
<p>JAVA_HOME: <%= javaHome %></p>
```

14. การรับข้อมูลจาก External API (HTTP Request)

JSP สามารถเรียกใช้ API ภายนอกโดยใช้ HttpURLConnection

(14.1) ตัวอย่างการดึงข้อมูลจาก API (weather.jsp)

```
<%@ page import="java.net.*, java.io.*" %>
<%
    URL url = new
URL("https://api.weatherapi.com/v1/current.json?key=YOUR_API_KEY&q=Bangkok");
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("GET");

    BufferedReader reader = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
    String line;
    StringBuilder response = new StringBuilder();
    while ((line = reader.readLine()) != null) {
        response.append(line);
    }
    reader.close();
%>
<p>Weather Data: <%= response.toString() %></p>
```

15. การรับข้อมูลจาก Database (JDBC)

JSP สามารถเชื่อมต่อกับฐานข้อมูลโดยใช้ JDBC

(15.1) ตัวอย่างการดึงข้อมูลจาก MySQL (db.jsp)

```
<%@ page import="java.sql.*" %>
<%
```

```

String url = "jdbc:mysql://localhost:3306/mydb";
String user = "root";
String pass = "password";

Connection conn = DriverManager.getConnection(url, user, pass);
Statement stmt = conn.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM users");

while (rs.next()) {
%>
    <p>User: <%= rs.getString("username") %>, Email: <%= rs.getString("email") %></p>
<%
    }
    conn.close();
%>

```

สรุปวิธีการรับข้อมูลใน JSP

วิธีการรับข้อมูล	คำอธิบาย	ตัวอย่าง
Hidden Fields	ส่งค่าผ่าน <input type="hidden">	request.getParameter("sessionID")
Multi-part Form Data	รับไฟล์จากฟอร์มอัปโหลด	ServletFileUpload
Servlet Context	รับค่าจาก web.xml	getServletContext().getInitParameter("appTitle")
WebSockets	รับข้อมูลแบบ real-time	@ServerEndpoint("/chat")
Request Dispatcher	ส่งข้อมูลระหว่างหน้า JSP	request.getRequestDispatcher("page.jsp")
Environment Variables	รับค่าจาก OS	System.getenv("JAVA_HOME")
External API	เรียก API ผ่าน HTTP	HttpURLConnection
Database (JDBC)	ดึงข้อมูลจากฐานข้อมูล	ResultSet rs = stmt.executeQuery("SELECT * FROM users");

นี่เป็นวิธีที่ครอบคลุมและหลากหลายในการรับข้อมูลใน JSP สามารถนำไปใช้ได้ตามความต้องการของเว็บแอปพลิเคชัน! □

คุณได้ครอบคลุมเกือบทุกวิธีการรับข้อมูลใน JSP แล้ว! แต่ยังมีบางวิธีเพิ่มเติมที่อาจเป็นประโยชน์ในบางสถานการณ์ เช่น:

16. การรับข้อมูลจาก Web Services (SOAP & REST)

นอกจาก HTTP API ทั่วไป JSP ยังสามารถเรียกใช้ **SOAP** หรือ **RESTful Web Services** ได้โดยใช้ **JAX-RS (REST)** หรือ **JAX-WS (SOAP)**

(16.1) การเรียก REST API ผ่าน HttpURLConnection

```
<%@ page import="java.net.*, java.io.*" %>
<%
    URL url = new URL("https://jsonplaceholder.typicode.com/todos/1");
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("GET");

    BufferedReader reader = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
    String line;
    StringBuilder response = new StringBuilder();
    while ((line = reader.readLine()) != null) {
        response.append(line);
    }
    reader.close();
%>
<p>API Response: <%= response.toString() %></p>
```

- ใช้ดึงข้อมูลจาก REST API
- สามารถแปลง JSON เป็น Java Object ด้วย org.json หรือ Gson

17. การรับข้อมูลจาก Webhooks

Webhooks ใช้รับข้อมูลที่ส่งมาจากระบบภายนอก เช่น **Stripe, GitHub, PayPal**

(17.1) การรับ Webhook Request ใน JSP

```
<%@ page import="java.io.*" %>
<%
    BufferedReader reader = request.getReader();
```

```

StringBuilder jsonInput = new StringBuilder();
String line;

while ((line = reader.readLine()) != null) {
    jsonInput.append(line);
}

out.println("Received Webhook: " + jsonInput.toString());
%>


- ใช้ request.getReader() เพื่อรับข้อมูลที่ถูกโพสต์มา
- เหมาะสำหรับรับ payment notifications, GitHub events ฯลฯ

```

18. การรับข้อมูลจาก Command Line Arguments

หากรัน JSP ภายใต้ **embedded servers** (เช่น Jetty หรือ Tomcat standalone) สามารถรับ command-line arguments ได้

(18.1) การอ่าน JVM Arguments (-Dproperty=value)

```

<%
    String appMode = System.getProperty("mode", "development");
%>
<p>Application Mode: <%= appMode %></p>


- รันเซิร์ฟเวอร์ด้วย java -Dmode=production -jar server.jar

```

19. การรับข้อมูลจาก Remote Database ผ่าน JNDI (Java Naming and Directory Interface)

JSP สามารถใช้ JNDI เพื่อเชื่อมต่อฐานข้อมูลที่กำหนดค่าในเซิร์ฟเวอร์ **Tomcat / GlassFish**

(19.1) การดึง Connection จาก JNDI

```

<%@ page import="javax.naming.*, javax.sql.*, java.sql.*" %>
<%
    Context initContext = new InitialContext();
    DataSource ds = (DataSource) initContext.lookup("java:comp/env/jdbc/MyDB");
    Connection conn = ds.getConnection();

    Statement stmt = conn.createStatement();
    ResultSet rs = stmt.executeQuery("SELECT * FROM users");

```

```

while (rs.next()) {
%>
    <p>User: <%= rs.getString("username") %></p>
<%
    }
    conn.close();
%>

```

- เหมาะสำหรับเว็บที่ใช้ Connection Pooling

20. การรับข้อมูลจาก Session Tracking (URL Rewriting)

ใช้ **URL Rewriting** เพื่อส่งค่าผ่านพารามิเตอร์ใน URL แทนการใช้ Cookies

(20.1) การสร้างลิงก์พร้อม Session ID

```

<a href="nextPage.jsp?<%= response.encodeURL("sessionID=12345") %>">Go to Next
Page</a>

```

(20.2) การรับค่าจาก URL

```

<%
    String sessionID = request.getParameter("sessionID");
%>
<p>Session ID: <%= sessionID %></p>

```

- เหมาะสำหรับรองรับเบราว์เซอร์ที่ปิด Cookies

21. การรับข้อมูลจาก Server Logs

JSP สามารถอ่าน Log Files เพื่อดึงข้อมูลย้อนหลัง

(21.1) การอ่านไฟล์ Log

```

<%@ page import="java.io.*" %>
<%
    File logFile = new File("/var/logs/app.log");
    BufferedReader reader = new BufferedReader(new FileReader(logFile));
    String line;

    while ((line = reader.readLine()) != null) {
%>
        <p><%= line %></p>
<%
    }

```

```

    }
    reader.close();
%>

```

- เหมาะสำหรับดึงข้อมูลจาก Access Logs หรือ Error Logs

22. การรับข้อมูลจาก User Authentication (SSO, OAuth, JWT)

JSP สามารถใช้ OAuth หรือ SSO เช่น **Google Login, Facebook Login, JWT**

(22.1) การอ่าน Header Authentication

```

<%
    String authHeader = request.getHeader("Authorization");
%>
<p>Auth Token: <%= authHeader %></p>

```

- ใช้ตรวจสอบ JWT หรือ Bearer Token

23. การรับข้อมูลจาก IoT Devices (MQTT, WebSockets)

หาก JSP ใช้ร่วมกับ IoT สามารถรับค่าจาก MQTT Broker ได้

(23.1) รับค่าจาก MQTT Broker

```
import org.eclipse.paho.client.mqttv3.*;
```

```

public class MQTTListener {
    public static void main(String[] args) throws Exception {
        MqttClient client = new MqttClient("tcp://mqtt.example.com:1883", "JSPClient");
        client.connect();
        client.subscribe("sensor/temperature", (topic, msg) -> {
            System.out.println("Temperature: " + new String(msg.getPayload()));
        });
    }
}

```

- ใช้รับค่าจาก IoT Sensors

24. การรับข้อมูลจาก Blockchain (Smart Contracts)

JSP สามารถอ่านค่าจาก Ethereum หรือ Hyperledger

(24.1) การอ่านค่าจาก Ethereum Smart Contract

```
Web3j web3 = Web3j.build(new HttpService("https://mainnet.infura.io/v3/YOUR_API_KEY"));
```

```
EthGetBalance balance = web3.ethGetBalance("0xYourEthereumAddress",
DefaultBlockParameterName.LATEST).send();
System.out.println("Balance: " + balance.getBalance());
```

- เหมาะสำหรับ DApps

25. การรับข้อมูลจาก Chatbots (Dialogflow, Telegram, WhatsApp)

JSP สามารถเชื่อมต่อ Chatbot APIs เพื่อรับข้อความจากผู้ใช้

(25.1) การรับข้อความจาก Telegram Bot

```
<%@ page import="java.net.*, java.io.*" %>
<%
    URL url = new URL("https://api.telegram.org/botTOKEN/getUpdates");
    HttpURLConnection conn = (HttpURLConnection) url.openConnection();
    conn.setRequestMethod("GET");

    BufferedReader reader = new BufferedReader(new
InputStreamReader(conn.getInputStream()));
    String line;
    StringBuilder response = new StringBuilder();
    while ((line = reader.readLine()) != null) {
        response.append(line);
    }
    reader.close();
%>
<p>Telegram Messages: <%= response.toString() %></p>
```

- ใช้ดึงข้อความจาก Telegram Bot

สรุปเพิ่มเติม

วิธีการรับข้อมูล	คำอธิบาย	ตัวอย่าง
SOAP / REST API	ดึงข้อมูลจาก Web Services	HttpURLConnection
Webhooks	รับ event จากระบบภายนอก	request.getReader()
Command-line Args	อ่านค่าที่ส่งมาตอนรัน	System.getProperty("mode")
JNDI	ดึง Connection จาก DataSource	context.lookup("jdbc/MyDB")

วิธีการรับข้อมูล	คำอธิบาย	ตัวอย่าง
MQTT / IoT	รับค่าจาก IoT Sensors	MqttClient.subscribe("topic")

วิธีเหล่านี้ช่วยให้ JSP สามารถเชื่อมต่อกับระบบอื่น ๆ ได้ในระดับ Enterprise! ☐

โปรเจกต์ JSP ที่มี Homepage ใช้ NetBeans + GlassFish Web Server

นี่คือตัวอย่างโครงสร้างโปรเจกต์ JSP ที่มี Homepage และ 5 หน้าหลัก ใช้ NetBeans + GlassFish Web Server

1. โครงสร้างโปรเจกต์

MyWebApp/

web/	
index.jsp	(หน้าแรก)
about.jsp	(เกี่ยวกับเรา)
services.jsp	(บริการ)
contact.jsp	(ติดต่อเรา)
gallery.jsp	(แกลเลอรี่)
styles.css	(ไฟล์ CSS)
menu.jsp	(เมนูนำทาง)
WEB-INF/	
web.xml	(ไฟล์ตั้งค่า)
src/	
build.xml	
nbproject/	
manifest.mf	

2. สร้างแต่ละไฟล์ JSP

(1) index.jsp – หน้าแรก

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ include file="menu.jsp" %>
<html>
<head>
  <title>Homepage</title>
```

```
<link rel="stylesheet" type="text/css" href="styles.css">
</head>
<body>
  <h1>ยินดีต้อนรับสู่เว็บไซต์ของเรา</h1>
  <p>นี่คือหน้าแรกของเว็บไซต์ JSP</p>
</body>
</html>
```

(2) menu.jsp – เมนูนำทาง

```
<div class="menu">
  <a href="index.jsp">หน้าหลัก</a> |
  <a href="about.jsp">เกี่ยวกับเรา</a> |
  <a href="services.jsp">บริการ</a> |
  <a href="gallery.jsp">แกลเลอรี่</a> |
  <a href="contact.jsp">ติดต่อเรา</a>
</div>
<hr>
```

(3) about.jsp – เกี่ยวกับเรา

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ include file="menu.jsp" %>
<html>
<head>
  <title>เกี่ยวกับเรา</title>
</head>
<body>
  <h2>เกี่ยวกับเรา</h2>
  <p>เราคือบริษัทที่ให้บริการเกี่ยวกับ JSP และ Java Web Development</p>
</body>
</html>
```

(4) services.jsp – บริการ

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ include file="menu.jsp" %>
```

```
<html>
<head>
  <title>บริการ</title>
</head>
<body>
  <h2>บริการของเรา</h2>
  <ul>
    <li>พัฒนาเว็บด้วย JSP</li>
    <li>เชื่อมต่อฐานข้อมูล MySQL</li>
    <li>ใช้ PrimeFaces สำหรับ UI</li>
  </ul>
</body>
</html>
```

(5) gallery.jsp – แกลเลอรี

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ include file="menu.jsp" %>
<html>
<head>
  <title>แกลเลอรี</title>
</head>
<body>
  <h2>แกลเลอรี</h2>
  <p>รวมภาพต่าง ๆ ที่เกี่ยวข้องกับบริการของเรา</p>
  
  
</body>
</html>
```

(6) contact.jsp – ติดต่อเรา

```
<%@ page contentType="text/html; charset=UTF-8" %>
<%@ include file="menu.jsp" %>
<html>
<head>
```

```
<title>ติดต่อเรา</title>
</head>
<body>
  <h2>ติดต่อเรา</h2>
  <form action="contact.jsp" method="post">
    ชื่อ: <input type="text" name="name"><br>
    อีเมล: <input type="email" name="email"><br>
    ข้อความ: <textarea name="message"></textarea><br>
    <input type="submit" value="ส่ง">
  </form>

  <%
    String name = request.getParameter("name");
    String email = request.getParameter("email");
    String message = request.getParameter("message");

    if (name != null && email != null && message != null) {
  %>
    <p><strong>ขอบคุณที่ติดต่อเรา, <%= name %>!</strong></p>
  <%
    }
  %>
</body>
</html>
```

(7) styles.css – ไฟล์ CSS

```
body {
  font-family: Arial, sans-serif;
  margin: 20px;
  padding: 20px;
}

.menu {
  background-color: #333;
  padding: 10px;
```

```

}
.menu a {
    color: white;
    text-decoration: none;
    margin-right: 15px;
}
.menu a:hover {
    text-decoration: underline;
}

```

3. ตั้งค่า web.xml

ไฟล์ web.xml อยู่ใน WEB-INF/ เพื่อกำหนดค่าการรันบน GlassFish

```

<web-app xmlns="http://java.sun.com/xml/ns/javaee" version="3.0">
    <display-name>My JSP Web Application</display-name>
    <welcome-file-list>
        <welcome-file>index.jsp</welcome-file>
    </welcome-file-list>
</web-app>

```

4. การตั้งค่า NetBeans + GlassFish

1) เปิด NetBeans และสร้างโปรเจกต์

- **File** → **New Project** → **Java Web** → **Web Application**
- ตั้งชื่อโปรเจกต์เป็น MyWebApp
- เลือก **GlassFish Server** เป็นเซิร์ฟเวอร์
- ใช้ **JSP** และ **Servlet**

2) เพิ่มไฟล์ JSP ลงใน web/

- คลิกขวา Web Pages → **New** → **JSP File**
- สร้าง index.jsp, about.jsp, services.jsp, gallery.jsp, contact.jsp, menu.jsp

3) เพิ่ม styles.css ลงใน web/

- คลิกขวา Web Pages → **New** → **Cascading Style Sheet**
- ตั้งชื่อเป็น styles.css

4) กำหนด GlassFish ให้รองรับ JSP

- คลิกขวาโปรเจกต์ → **Properties**

- ไปที่ **Run** และตรวจสอบว่า Server: GlassFish
- ไปที่ **Libraries** → เพิ่ม JSTL (JavaServer Pages Standard Tag Library)

5) Run Project บน GlassFish

- คลิกขวา MyWebApp → **Run**
- เปิดเบราว์เซอร์และไปที่ <http://localhost:8080/MyWebApp/>

5. สรุป

- ✓ โครงสร้าง 5 หน้า JSP
- ✓ เมนูนำทาง menu.jsp
- ✓ ฟอर्मรับข้อมูล contact.jsp
- ✓ ใช้ GlassFish Web Server
- ✓ ใช้ NetBeans เพื่อพัฒนา
- ☐ หลังจากรันโปรเจกต์แล้ว จะได้เว็บไซต์ JSP ที่ใช้งานได้จริง! ☐

สรุป

ในยุคที่เทคโนโลยีเว็บพัฒนาอย่างต่อเนื่อง JSP (JavaServer Pages) ได้กลายเป็นหนึ่งในเครื่องมือสำคัญในการสร้างเว็บแอปพลิเคชันแบบไดนามิกด้วยภาษา Java โดยเฉพาะเมื่อใช้งานร่วมกับ NetBeans IDE และ GlassFish Web Server ผู้พัฒนาสามารถออกแบบระบบที่มีความยืดหยุ่นและปลอดภัย หนังสือเล่มนี้จึงนำเสนอองค์ประกอบหลักของไฟล์ JSP อย่างเป็นระบบ ไม่ว่าจะเป็น Directives, Declarations, Scriptlets, Expressions รวมถึงการใช้ Implicit Objects และ Standard Actions ต่าง ๆ พร้อมตัวอย่างการใช้งานจริง เพื่อช่วยให้ผู้อ่านเข้าใจการทำงานของ JSP ได้อย่างชัดเจน

เนื้อหายังครอบคลุมการใช้ JSTL (JavaServer Pages Standard Tag Library), Expression Language (EL), และ Custom Tags ที่ช่วยให้โค้ดอ่านง่ายและแยกส่วนของตรรกะออกจากการแสดงผลได้อย่างชัดเจน นอกจากนี้ยังกล่าวถึงการจัดการข้อมูลจากผู้ใช้ การจัดการข้อผิดพลาด และการสร้างโปรเจกต์ JSP ที่มีหน้า Homepage แบบครบวงจรบน NetBeans และ GlassFish Web Server โดยมีคำอธิบายเป็นลำดับขั้น เหมาะสำหรับทั้งผู้เริ่มต้นและผู้ที่ต้องการต่อยอดความรู้ในการพัฒนาเว็บแอปพลิเคชันด้วย JSP อย่างมีประสิทธิภาพ